Building an Urdu BPE Tokenizer from Scratch

CS 417 Large Language Models · Assignment 1 · NUST SEECS, Spring 2026
[Name 1 / Reg. No.] · [Name 2 / Reg. No.]

Setup in brief

We collected **1.30 GB** of cleaned Urdu text from four sources: VOA Urdu (news, scraped by us), Urdu Wikipedia, Urdu Wikisource (literature) and MADLAD-400 (web). The full details are in `DATA_CARD.md`. We trained a byte-level BPE tokenizer, written in plain Python, on the 1,276 MB `train.txt` to exactly **50,000 tokens**: 256 bytes, then `<|endoftext|>` as ID 256, then 49,743 merges.

Training (fast trainer, full train.txt)	307 s wall-clock on an Intel Core Ultra 7 155U (12 cores, 15.4 GB RAM), Python 3.14
Encoded corpus	`train.bin` 206.3M tokens, `val.bin` 2.0M tokens (flat `uint16`)
Fast vs reference trainer	Identical merges on a 1 MB sample of our corpus (from `test.txt`) for the first 5,000 merges. The fast trainer took 2.0 s and the reference took 1,578 s (about 26 minutes), roughly 790 times slower.

Pre-tokenization pattern. `?[\p{L}\p{M}\u200c]+| ?\p{N}{1,3}| ?[^\s\p{L}\p{M}\p{N}]+|\s+(?!\S)|\s+`
The first branch treats letters, combining marks (`\p{M}`: zer, zabar, pesh, shadda) and ZWNJ as one word. So اُردُو stays a single piece instead of breaking at every diacritic, as it does with GPT-2's pattern. As in GPT-2, a leading space belongs to the word that follows it. Numbers are split into chunks of at most three digits, so the tokenizer can't spend merges memorizing long numbers. Every character matches some branch, which keeps encoding lossless. Normalization (Arabic → Urdu letters, all digits → ASCII, ZWNJ kept only between letters) happens before training and is not part of the tokenizer.

Q1. What did the merges learn?

Merges 1–50

␣ marks a leading space. ⟨…⟩ marks a token that is not valid UTF-8 on its own, shown as hex bytes.

#	Token	#	Token	#	Token	#	Token	#	Token
1	⟨20 d8⟩	11	ن	21	یں	31	ت␣	41	ع
2	ی	12	ا␣	22	س␣	32	کی␣	42	و␣
3	ا	13	ل	23	ب␣	33	پ␣	43	ق
4	ک	14	ت	24	د	34	ن␣	44	ہے␣
5	⟨20 d9⟩	15	م	25	ھ	35	ار	45	ف
6	ر	16	م␣	26	۔	36	ان	46	اس␣
7	و	17	س	27	ب	37	ج␣	47	اور␣
8	ہ	18	ں	28	گ	38	میں␣	48	ل␣
9	ے	19	⟨d8 a7 d8⟩	29	کے␣	39	ٹ	49	ائ
10	ک␣	20	ہ␣	30	ور	40	د␣	50	ری

Every 5,000th merge

#	5,000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	49,743
Token	مسکر␣	فرینڈ␣	ولاد␣	محافظوں␣	ged	تیندوے␣	372␣	وہیں	سمبل␣	چانڈیو␣

Rebuilding letters. Every Urdu letter is two bytes in UTF-8, so most early merges just put letters back together: **22 of the first 50** merges join two bytes into one letter, starting with ی, ا and ک. The order follows letter frequency. Rare letters come much later (ڑ at merge 1,177), and so do diacritics (zer at 164, pesh at 225). Over the whole vocabulary, 325 merges rebuild a two-byte character.

Invalid UTF-8 tokens. Only **3 of the first 50** merges (1, 5 and 19) and **208 of all 49,743** (0.4%) are not valid UTF-8 on their own. Merge 1 is a space plus the byte `d8`, which is the first half of most common letters. Nearly every Urdu word

starts with "space + d8/d9", so this pair is counted more often than any complete letter. These half-letters are stepping stones: merge 12 (ا) is built from merge 1, and merge 49 (ائ) from merge 19. The later invalid tokens are fragments of 3-byte characters such as curly quotes and dashes (`e2 80`), plus some Hindi and Bengali script that leaked in through Wikipedia.

First whole word. Merge **29** gives کے ("of"), which appears 61,451 times in `test.txt`. It's followed soon after by کی (32), میں (38), ہے (44) and اور (47). These are the postpositions and helper verbs that hold every Urdu sentence together. By merge 5,000 the tokenizer is adding full content words. By the last merges it is adding rare names such as چانڈیو (Chandio), which shows the vocabulary has started to fit corpus-specific words.

The 20 longest tokens

Merge	Token	Chars	Merge	Token	Chars
40,417	hadiththransmitters	19	49,101	Nottinghamshire	16
36,859	shiaonlinelibrary	18	3,083	CricketArchive	15
37,216	theatreworldawards	18	4,728	snaccooperative	15
40,831	islamansiklopedisi	18	10,215	انسائیکلوپیڈیا	15
2,692	minorplanetcenter	17	18,763	Administration	15
15,494	githubusercontent	17	19,888	Transportation	15
26,115	----------	16	20,864	Internationals	15
31,975	*-*-*-*-*-*-*-*-	16	21,507	cryptocurrency	15
35,132	Recommendations	16	23,549	Synchronkartei	15
40,166	Gloucestershire	16	28,865	frauendatenbank	15

Measured in characters, the longest tokens are mostly English, and most of them aren't ordinary words. They are pieces of website names from Wikipedia's reference sections (`githubusercontent`, `minorplanetcenter`), repeated on thousands of pages, plus separator lines made of - and `*-`. The longest real Urdu tokens are loanwords such as انسائیکلوپیڈیا (encyclopedia) and انٹرکانٹینینٹل (intercontinental). Each Urdu character costs two bytes, so the same merge budget covers fewer Urdu characters. The table shows where the corpus is still noisy: reference boilerplate in Wikipedia uses up some merges.

Q2. How does it compare with existing tokenizers?

Tokens per word, where words are counted by splitting on whitespace. Lower is better. `test.txt` is our held-out 1% split (2,880 documents). The seven samples are short passages we wrote or picked by hand, and they're in `analysis/samples/`.

Input	Words	Ours	GPT-2 (`gpt2`)	`o200k_base`
test.txt	1,701,409	**1.29**	5.77	1.71
News	68	**1.06**	6.10	1.35
Poetry (Ghalib, Iqbal)	51	**1.14**	5.61	1.57
Urdu with English words	43	1.37	3.88	**1.16**
Roman Urdu	34	2.56	2.32	**1.97**
Numbers and dates	44	**1.52**	5.27	1.86
Text with diacritics	29	**1.83**	7.48	2.83
Personal names	33	**1.48**	7.48	2.36

On ordinary Urdu our tokenizer is clearly the most efficient. It uses 1.29 tokens per word on `test.txt`. `o200k_base` needs 33% more tokens, and GPT-2 needs about **4.5 times as many**, because its English-trained vocabulary has almost no Urdu merges and falls back to bytes. Our news and poetry samples are close to one token per word. This matters for Assignment 2: the same context window will hold several times more Urdu text.

The advantage disappears outside Urdu script. With English words mixed in, and especially in Roman Urdu, `o200k_base` wins, and for Roman Urdu even GPT-2 beats us. Our 50,000 tokens were spent almost entirely on Urdu, while the other two tokenizers were trained on huge amounts of Latin-script text. Diacritics and names cost us more than plain news, but we still use far fewer tokens than both other tokenizers there.

Q3. Three inputs our tokenizer handles badly

Below, each token is shown in its own box. Red boxes are byte fragments that are not valid UTF-8 on their own.

1. Roman Urdu: 2.64 tokens/word (the same sentence in Urdu script: 1.00)

M | uj | he | ▁Urdu | ▁b | ol | na | ▁b | oh | at | ▁p | as | and | ▁h | ai | ▁le | kin | ▁l | ikh | na | ▁m | ush | k | il | ▁l | ag | ta | ▁h | ai

"Mujhe Urdu bolna bohat pasand hai lekin likhna mushkil lagta hai" becomes 29 tokens. The same sentence in Urdu script becomes 11, one per word. Our cleaning kept only Arabic-script lines, so the tokenizer saw almost no Roman Urdu (a gap noted in the data card). The few Latin merges it learned come from English names and references in Wikipedia. So common Roman Urdu words such as *bohat*, *hai* and *mushkil* are split into 2–4 pieces. `o200k_base` does better here (1.82 tokens/word).

2. Text typed with Arabic letters and digits: 2.80 tokens/word (after normalization: 1.07)

پا | d9 | 83 | ستان | 20 db | b1 | db | b4 | اگست▁ | 20 db | b1 | db | b9 | db | b4 | db | b7 | 20 d9 | 83 | و | آزاد▁ | 20 d9 | 87 | وا | …

This sentence, "پاکستان ۱۴ اگست ۱۹۴۷ کو آزاد ہوا...", was typed with Arabic kaf ك, heh ه, yeh ي and Eastern-Arabic digits. That's common when someone uses an Arabic keyboard. These characters look the same as the Urdu ones, but they are different code points. Our normalizer maps all of them to Urdu letters and ASCII digits before training, so the tokenizer never saw them and learned no merges for them. Each one falls back to two raw bytes and breaks the word around it, so پاکستان becomes four tokens instead of one. The fix is to run `normalize()` before `encode()`, as our pipeline does. The tokenizer itself must stay lossless, so it doesn't normalize.

3. Emoji: 5.33 tokens/word (the same text without emoji: 1.00)

بہت	خوب	20 f0 9f 98	82	f0 9f	98	82	f0 9f	94	a5	f0 9f	91	8d	f0 9f	8f	bd

"بہت خوب 😂😂🔥👍🏽" uses 2 tokens for the words and 14 for four emoji. Each emoji is 4 bytes in UTF-8, and a thumbs-up with a skin tone is two code points (8 bytes). Our corpus is edited news, encyclopedia and literature text, which contains almost no emoji. So only a few shared byte prefixes such as `f0 9f` became merges, and each emoji still costs 2–3 tokens of meaningless byte fragments. Nothing is lost, and decoding still gives back the exact text. But chat-style Urdu would be expensive to model. `o200k_base`, trained on web and chat data, uses 10 tokens for the same input.

Reproduce: `python -m analysis.merges_report` (Q1), `python -m analysis.compare_tokenizers` (Q2), `python -m analysis.failure_cases` (Q3), `python -m tests.compare_trainers --input data/final/test.txt --merges 5000` (trainer check).